

[ChatGPT, 9.27.26] *"Please generate a IPv4 Parsing script named "extractIPv4" for use in the C programming language. Requirements: Must optionally be able to catch a 4 digit port indicated by a ':' symbol. If port has 5+ digits, the entire set of parsed characters must be disregarded. Reject anything with an incorrect octet count, an empty octet, an out-of-range octet or port, a disallowed leading zero, a second colon, a colon not immediately after the fourth octet, or stray period/colons directly adjacent to an address that is otherwise valid."*

Below is a self-contained C implementation of `extractIPv4`. It scans arbitrary text for the first valid IPv4 address, optionally followed immediately by `:` and a **1–4 digit port**.

It rejects:

- Wrong octet count
- Empty octets
- Octets outside `0–255`
- Leading zeros such as `01` or `001`
- Ports longer than 4 digits
- Ports outside `1–9999`
- A second colon
- A colon anywhere except immediately after the fourth octet
- Extra `.` or `:` directly adjacent to an otherwise valid address
- Partial matches inside a larger numeric/dotted/colon sequence

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdbool.h>
```

```
#define IPV4_MAX_LEN 15
#define PORT_MAX 9999
```

```

/*
 * Extract the first valid IPv4 address from 'input'.
 *
 * On success:
 *   ip_out receives the IPv4 address, NUL terminated.
 *   port_out receives the port, or -1 if no port was present.
 *
 * Returns true on success, false if no valid IPv4 address was found.
 *
 * Examples accepted:
 * "192.168.1.1"
 * "server=192.168.1.1:8080"
 * "IP: 10.0.0.1:443"
 *
 * Examples rejected:
 * "192.168.01.1"    leading zero
 * "192.168.1"       wrong octet count
 * "192.168.1.256"   out of range
 * "192.168..1.1"    empty octet
 * "192.168.1.1:12345" 5-digit port
 * "192.168.1.1::80"  second colon
 * "192.168.1.1."    stray period
 * "192.168.1.1:80:" stray colon
 */
bool extractIPv4(const char *input, char *ip_out, size_t ip_out_size,
                 int *port_out)
{
    if (!input || !ip_out || !port_out || ip_out_size < 16)
        return false;

    *port_out = -1;
    ip_out[0] = '\0';

    const char *p = input;

    while (*p) {
        /*
         * A candidate must begin with a digit.
         * This also prevents starting in the middle of an octet.
         */
        if (!isdigit((unsigned char)*p)) {
            p++;
            continue;
        }
    }

```

```

const char *start = p;
unsigned int octets[4];
int octet_count = 0;
bool valid = true;

/*
 * The character immediately before the candidate cannot be
 * a digit, '.' or ':'; otherwise this would be a partial
 * extraction from a larger malformed address.
 */
if (start != input) {
    unsigned char prev = (unsigned char)start[-1];

    if (isdigit(prev) || prev == '.' || prev == ':') {
        p++;
        continue;
    }
}

while (octet_count < 4) {
    unsigned int value = 0;
    int digits = 0;

    /* Empty octet. */
    if (!isdigit((unsigned char)*p)) {
        valid = false;
        break;
    }

    /*
     * Leading zero is permitted only for the single digit "0".
     */
    if (*p == '0' && isdigit((unsigned char)p[1])) {
        valid = false;
        break;
    }

    while (isdigit((unsigned char)*p)) {
        /*
         * More than three digits can never be a valid IPv4
         * octet. Consume the candidate as invalid rather than
         * accepting a shorter prefix.
         */

```

```

    if (digits >= 3) {
        valid = false;

        while (isdigit((unsigned char)*p))
            p++;

        break;
    }

    value = value * 10u + (unsigned int)(*p - '0');
    digits++;
    p++;
}

if (!valid)
    break;

if (value > 255 || digits == 0) {
    valid = false;
    break;
}

octets[octet_count++] = value;

if (octet_count == 4)
    break;

/*
 * Every separator between octets must be exactly one '!'.
 */
if (*p != '!') {
    valid = false;
    break;
}

p++;

/* Reject an empty octet such as "1.2.3..4". */
if (!isdigit((unsigned char)*p)) {
    valid = false;
    break;
}
}

```

```

/*
 * Must have exactly four octets.
 */
if (!valid || octet_count != 4) {
    p = start + 1;
    continue;
}

/*
 * The character after the fourth octet determines what follows.
 *
 * '.' is forbidden because it would mean the address is part
 * of a larger dotted sequence.
 *
 * ':' is allowed only here, immediately after octet four.
 *
 * A digit is impossible here because the fourth-octet parser
 * consumed all consecutive digits.
 */
const char *after_ip = p;

if (*after_ip == '.') {
    p = start + 1;
    continue;
}

int port = -1;

if (*after_ip == ':') {
    const char *port_start = after_ip + 1;
    const char *q = port_start;
    unsigned int port_value = 0;
    int port_digits = 0;

    /*
     * Port must contain at least one digit.
     */
    if (!isdigit((unsigned char)*q)) {
        p = start + 1;
        continue;
    }

    /*
     * Parse the port. Five or more digits invalidate the

```

```

* entire candidate, rather than accepting its first four.
*/
while (isdigit((unsigned char)*q)) {
    if (port_digits >= 4) {
        valid = false;

        /*
         * Consume the entire digit sequence so a 5-digit
         * port cannot accidentally be partially re-matched.
         */
        while (isdigit((unsigned char)*q))
            q++;

        break;
    }

    /*
     * Disallow leading zeros in multi-digit ports.
     */
    if (port_digits == 0 && *q == '0' &&
        isdigit((unsigned char)q[1])) {
        valid = false;

        while (isdigit((unsigned char)*q))
            q++;

        break;
    }

    port_value = port_value * 10u +
        (unsigned int)(*q - '0');

    port_digits++;
    q++;
}

if (!valid) {
    p = start + 1;
    continue;
}

/*
 * Port range: 1..9999.
 */

```

```

    if (port_digits == 0 || port_value == 0 ||
        port_value > PORT_MAX) {
        p = start + 1;
        continue;
    }

    /*
     * A second colon immediately following the port is
     * explicitly forbidden.
     */
    if (*q == ':') {
        p = start + 1;
        continue;
    }

    /*
     * A period immediately following the port is also
     * considered a stray delimiter.
     */
    if (*q == '.') {
        p = start + 1;
        continue;
    }

    port = (int)port_value;
    after_ip = q;
}

/*
 * If the address had no port, a colon here would have been
 * handled above. Any '.' was already rejected.
 *
 * Also prevent extraction from something such as:
 * 192.168.1.1:80:90
 * 192.168.1.1.5
 */
if (*after_ip == ':' || *after_ip == '.') {
    p = start + 1;
    continue;
}

/*
 * Copy the validated IPv4 address.
 */

```

```
size_t ip_len = (size_t)(p - start);

if (ip_len >= ip_out_size) {
    return false;
}

memcpy(ip_out, start, ip_len);
ip_out[ip_len] = '\0';

*port_out = port;
return true;
}

return false;
}
```

One interpretation worth making explicit: this implementation treats **0** as a valid octet, but rejects **00**, **01**, etc.; for ports it allows **1–9999** and likewise rejects leading-zero forms such as **080**.